

Code Python — Mall Customers

Segmentation des clients · Machine Learning

*Réalisé par :
El alouy el amerany Aya
Limour Oumaima
Bakkali Dina
Mamoun Nour*

```
# =====
# IMPORTATIONS
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.cluster import KMeans

# =====
# 1. CHARGEMENT DU DATASET
# =====
df = pd.read_csv(r"C:\Users\lonovo\Downloads\Mall_Customers (1).csv")

# =====
# 2. EXPLORATION
# =====
print("=== PREMIÈRES LIGNES (.head) ===")
print(df.head())
print("\n=== INFORMATIONS GÉNÉRALES (.info) ===")
df.info()
print("\n=== STATISTIQUES (.describe) ===")
print(df.describe())

# =====
# 3. PRÉPARATION DES DONNÉES
# =====
# Vérification des valeurs nulles et doublons
print("\nValeurs nulles :")
print(df.isnull().sum())
print("\nNombre de doublons :")
print(df.duplicated().sum())

# Suppression des doublons
```

```

df = df.drop_duplicates()

# Encodage de la variable catégorielle Gender
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
print("\nColonne Gender après encodage :")
print(df['Gender'].head())

# Normalisation des variables numériques
scaler = StandardScaler()
cols = ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
df[cols] = scaler.fit_transform(df[cols])

print("\nDataset après préparation :")
print(df.head())
print("\nInformations finales :")
df.info()

# =====
# 4. APPRENTISSAGE SUPERVISÉ — RÉGRESSION
# =====
X = df.drop(columns=['CustomerID', 'Spending Score (1-100)'])
y = df['Spending Score (1-100)']
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=200, max_depth=15, random_state=42,
n_jobs=-1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("MSE :", mse)
print("RMSE :", np.sqrt(mse))
print("R² :", r2)

# Réel vs Prédit
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, alpha=0.4)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--")
plt.xlabel("Valeurs réelles")
plt.ylabel("Valeurs prédites")
plt.title("Réel vs Prédit")
plt.tight_layout()
plt.show()

# Importance des variables
importances = pd.Series(model.feature_importances_, index=X.columns)
importances.sort_values().plot(kind="barh")

```

```

plt.title("Importance des variables")
plt.tight_layout()
plt.show()

# Distribution des résidus
residus = y_test.values - y_pred
sns.histplot(residus, bins=50, kde=True)
plt.title("Distribution des Résidus")
plt.tight_layout()
plt.show()

# =====
# 5. APPRENTISSAGE NON SUPERVISÉ — K-MEANS
# =====
# Choix des variables pour le clustering
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Méthode du coude
inertia = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(K, inertia, marker='o')
plt.title('Méthode du coude')
plt.xlabel('Nombre de clusters')
plt.ylabel('Inertia')
plt.grid(True)
plt.show()

# Application de K-Means (k=5)
kmeans = KMeans(n_clusters=5, random_state=42)
df['Cluster'] = kmeans.fit_predict(X)
print("\nClusters trouvés :")
print(df[['Annual Income (k$)', 'Spending Score (1-100)', 'Cluster']].head())

# Visualisation des clusters
plt.figure(figsize=(8, 6))
plt.scatter(df['Annual Income (k$)'], df['Spending Score (1-100)'], c=df['Cluster'])
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s=200, marker='X')
plt.title('Segmentation des clients avec K-Means')
plt.xlabel('Revenu annuel')
plt.ylabel('Spending Score')
plt.grid(True)
plt.show()

```